

Vector Embeddings — From First Principles

Coder Army

1. What Can a Computer Actually Understand?

Suppose we give a computer the word:

Batman

For a human, this one word immediately activates many associations:

- Superhero
- Gotham
- Bruce Wayne
- DC
- Dark
- Serious
- Crime
- Vigilante
- Action
- Rich

A computer does not naturally have these associations.

At the lowest level, a computer ultimately works with numerical or binary representations. It can store the characters or tokens making up the word `Batman`, but storing symbols is not the same as understanding their meaning.

Representing symbols is not the same as representing meaning.

A computer can store:

B
a
t
m
a
n

Coder Army

perfectly without knowing what Batman actually represents.

2. Identity vs Meaning

Traditional computer systems are very good at exact matching.

```
"Batman".equals("Batman")
```

Result:

```
true
```

But:

```
"Batman".equals("Superman")
```

Result:

```
false
```

This tells us whether two strings are identical.

It does not tell us whether they are conceptually related.

Consider:

```
"I forgot my password."
```

```
"How can I recover my login credentials?"
```

These sentences are very different as strings.

But in meaning, they represent almost the same intent.

A system based purely on exact matching sees:

```
different
```

A human sees:

```
almost the same meaning
```

This gives us the central problem:

How do we convert meaning into something mathematics can operate on?

3. Why Do We Need Numbers?

Machine-learning models, neural networks, and mathematical algorithms fundamentally operate on numbers.

There is not much useful mathematics we can directly perform on:

```
Batman  
Superman  
Interstellar  
Pizza
```

But once something becomes numbers, we can perform operations such as:

```
addition  
multiplication  
distance  
angles  
probabilities  
matrix operations
```

So our first requirement is:

Turn real-world things into numerical representations.

Those real-world things could be:

- Words
- Sentences
- Documents
- Movies
- Users
- Songs
- Products
- Images
- Proteins
- Web pages

This problem existed long before modern LLMs.

Simply assigning a random number to every object does not solve the problem because the number would not represent any meaningful relationship between objects.

4. First Attempt: Represent Something Using One Number

Forget language for a moment.

Imagine we are building a movie recommendation system.

We could start by assigning every movie an **action score**.

Movie	Action Score
Avengers	10
Batman	9

Movie	Action Score
Interstellar	6
Titanic	2
Hera Pheri	1

Previously, these were only names.

Now we have:

```
Batman      → 9
Avengers    → 10
Titanic     → 2
```

For the first time, we have converted one characteristic of a movie into mathematics.

Similarity now becomes measurable

Suppose we want to know which movie is closer to Batman in terms of action.

```
Batman      = 9
Avengers    = 10
Titanic     = 2
```

Distance:

$$|9 - 10| = 1$$

$$|9 - 2| = 7$$

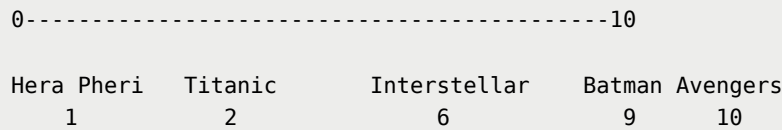
Therefore, in terms of action:

```
Batman is closer to Avengers than Titanic.
```

The computer can now mathematically calculate similarity.

A One-Dimensional Space

We have effectively created a one-dimensional world.



Every movie now has a position.

That position represents one feature:

Action

Once objects have positions inside a space:

Distance becomes meaningful.

The Limitation of One Number

Suppose:

Titanic = 2
Hera Pheri = 2

If action is our only feature, the system may conclude that these movies are highly similar.

But they clearly are not.

One is a romantic drama while the other is primarily comedy.

The problem is simple:

One number can capture only one property.

Real-world objects contain many properties.

5. From One Number to Multiple Numbers

Instead of representing only action, we can add another characteristic such as comedy.

Movie	Action	Comedy
Batman	9	1
Avengers	10	7
Titanic	2	1
Hera Pheri	2	10
Interstellar	6	1

Batman is no longer represented as:

9

It becomes:

[9, 1]

Titanic:

[2, 1]

Avengers:

[10, 7]

This ordered collection of numbers is called a **vector**.

6. What Is a Vector?

A useful machine-learning definition is:

A vector is an ordered collection of numbers representing something.

Example:

```
Batman = [9, 1]
```

where:

```
position 1 → action  
position 2 → comedy
```

Two dimensions are still unlikely to capture everything about a movie.

We could create:

```
Batman = [  
    9, // action  
    1, // comedy  
    2, // romance  
    9, // darkness  
    8, // violence  
    7  // mystery  
]
```

Batman is now represented using six characteristics.

Why Is Order Important?

These vectors are different:

```
[9, 2]
```

and:

```
[2, 9]
```

Suppose:

```
dimension 1 = action  
dimension 2 = comedy
```

Then:

```
[9, 2]
```

means:

```
high action  
low comedy
```

while:

```
[2, 9]
```

means:

```
low action  
high comedy
```

The position of every value matters because every dimension has a consistent role.

7. What Is a Dimension?

Each value in a vector represents one dimension.

```
[9]
```

has:

1 dimension

[9, 2]

has:

2 dimensions

[9, 2, 3]

has:

3 dimensions

A vector can also contain hundreds or thousands of dimensions.

For example:

768 dimensions

Humans cannot visualize a 768-dimensional space, but mathematics can still operate on it.

High-dimensional spaces are difficult for humans to visualize, not difficult for computers to calculate with.

8. Vector Space

A collection of vectors exists inside a **vector space**.

In two dimensions, we can imagine:

Coder Army

```
Batman    = [9, 1]
Titanic   = [2, 1]
Avengers  = [10, 7]
```

These numbers give every movie a location inside a mathematical space.

Once objects have positions, we can start asking:

```
Which objects are close?
Which objects are far apart?
```

This introduces a major shift.

Traditional systems often ask:

```
same or different?
```

Vector systems allow us to ask:

```
how similar?
```

Instead of only:

```
true
false
```

we can work with continuous similarity.

```
0.92 similarity
0.71 similarity
0.22 similarity
```

This idea is fundamental to vector search.

9. Measuring Similarity Between Vectors

Once objects exist inside a mathematical space, we need ways to compare them.

Three important approaches are:

1. Euclidean Distance
2. Dot Product
3. Cosine Similarity

The important part is understanding the intuition behind them rather than memorizing formulas.

10. Euclidean Distance

Euclidean distance is the normal idea of straight-line distance.

Suppose:

```
Batman    = [9, 1]
Avengers  = [10, 7]
Titanic   = [2, 10]
```

We can calculate the geometric distance between these points.

In general:

```
smaller distance → closer
larger distance  → farther apart
```

If our dimensions represent meaningful characteristics such as:

```
action
comedy
```

then two movies close together have similar values across those dimensions.

Therefore:

Geometric closeness can represent semantic similarity when the vector space itself has been constructed meaningfully.

Distance itself is not magic.

The representation matters.

Bad Representation → Bad Similarity

Suppose we represent movies using:

```
[
  number_of_letters_in_title,
  release_month
]
```

Batman might accidentally appear close to Titanic.

The distance calculation could be mathematically correct but semantically useless.

Therefore:

The usefulness of vector similarity depends heavily on the quality of the representation.

11. Why Direction Can Matter

Consider two user preference vectors:

```
User A = [9, 1]
User B = [90, 10]
```

Suppose the dimensions are:

```
[action preference, romance preference]
```

Their absolute values are very different.

But the ratio is identical:

```
9 : 1  
90 : 10
```

Both users strongly prefer action compared with romance.

Euclidean distance would say that they are far apart.

But geometrically their vectors point in the same direction.

For some applications, the **pattern of preferences** may matter more than the absolute magnitude.

This is where cosine similarity becomes useful.

12. Dot Product

Suppose:

```
A = [2, 3]  
B = [4, 5]
```

The dot product is:

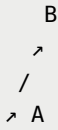
```
(2 × 4) + (3 × 5)
```

The dot product contains information about both:

- Direction
- Magnitude

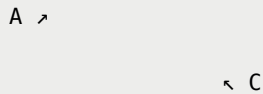
Vectors pointing in similar directions generally produce stronger positive alignment.

The geometric intuition is more important than the arithmetic:



These vectors point in similar directions.

Compare that with:



The directions are very different.

13. Cosine Similarity

Cosine similarity asks:

How similar is the direction of these two vectors?

It focuses heavily on the angle between vectors rather than their absolute size.

Typical interpretation:

```
1 → same direction
0 → unrelated / perpendicular directions
-1 → opposite directions
```

The practical behavior depends on how the embedding model and vectors are designed or normalized, but the main intuition remains:

Similar directions can represent similar patterns.

Recommendation-System Intuition

Imagine we have:

User preference vector ↗

Movie A ↗

Movie B ↑

Movie C ←

Movie A points most closely in the same direction as the user's preference vector.

Therefore:

```
similarity(User, Movie A) → high
```

The system can recommend Movie A.

At this point we have already built the basic intuition behind a recommendation engine without requiring an LLM.

14. Comparing Common Similarity Approaches

Metric	Main Idea	Useful Intuition
Euclidean Distance	Measures straight-line distance	How physically close are the vectors?
Cosine Similarity	Measures similarity in direction	Do the vectors represent a similar pattern regardless of size?
Dot Product	Combines direction and magnitude	Are the vectors aligned, and how strong are they?

The correct choice depends on how the vectors were created and what the application is trying to measure.

15. The Big Problem: Who Decides the Dimensions?

Until now, we manually created vectors such as:

```
Batman = [  
    action,  
    romance,  
    comedy,  
    darkness  
]
```

For example:

```
Batman = [9, 2, 3, 9]
```

But who decided these four dimensions?

We did.

That becomes a serious limitation.

16. Hand-Crafted Features

Manually designing useful characteristics is commonly known as **feature engineering**.

For movies, we might create dimensions such as:

```
action  
romance  
comedy  
horror  
violence  
family  
science-fiction  
mystery  
dialogue-heavy  
emotional  
dark  
historical
```

Values can then be manually or algorithmically assigned.

Historically, a large amount of machine learning relied on feature engineering.

It can work extremely well.

But it has major limitations.

17. Problem 1: How Many Features Should We Create?

Consider:

Interstellar

Should we represent:

space
science
family
fatherhood
time
love
survival
philosophy
physics
sadness
adventure
visual spectacle
hope
sacrifice
isolation

Different people may describe the same movie using completely different characteristics.

There could be thousands of meaningful properties.

Humans would have to decide which ones matter.

18. Problem 2: Features Depend on the Task

The same movie may require completely different features depending on what we are trying to solve.

Movie recommendation

Useful features might include:

```
genre
tone
pace
```

Box-office prediction

Useful features might include:

```
budget
actor popularity
release date
marketing spend
```

Parental controls

Useful features might include:

```
violence
language
sexual content
```

Same object.

Different representations.

There is no universally correct human-designed feature list.

19. Problem 3: Language Makes Feature Engineering Extremely Difficult

Consider the word:

King

We might invent features such as:

```
human
male
royalty
authority
wealth
adult
leader
politics
history
power
```

Now consider:

Queen

Many dimensions would overlap.

But then consider:

Apple

We suddenly need concepts such as:

```
fruit
food
company
technology
sweet
red
round
```

Then:

Java

could mean:

```
programming language
island
coffee
```

Our manually designed feature list starts becoming impossible to maintain.

20. Context Changes Meaning

Consider:

bank

Sentence 1:

"I deposited money at the bank."

Sentence 2:

"The fisherman sat on the river bank."

The word is identical.

The meaning is different.

Human language contains:

- Ambiguity
- Context
- Grammar

- Tone
- Intent
- Metaphor
- Relationships

Trying to manually design features for everything in language becomes almost impossible.

21. The Key Question

We know vectors are useful.

We want:

```
Batman    → vector
Superman  → vector
Titanic   → vector

king      → vector
queen     → vector
dog       → vector
```

And ideally we want the resulting space to behave like this:

```
Batman ≈ Superman

dog ≈ puppy

king ≈ queen
```

while:

```
Batman ≠ banana

dog ≠ database

king ≠ refrigerator
```

But we do not want humans to manually design thousands of dimensions.

The scalable solution is:

Let the machine learn the vector space itself.

This brings us to embeddings.

22. What Is an Embedding?

Suppose:

```
Batman    → vector A
Superman  → vector B
Titanic   → vector C
```

We want:

```
similarity(A, B) > similarity(A, C)
```

The important idea is:

We do not necessarily care what every individual number means. We care that the overall space behaves meaningfully.

This is the central idea behind embeddings.

23. Where Do Embedding Numbers Come From?

Suppose we see:

```
Batman = [0.23, -0.81, 0.48]
```

Who decided these values?

Not a human assigning explicit semantic scores.

The values are learned during training.

To understand this, start from random numbers.

Suppose our vocabulary contains only:

```
king  
queen  
banana  
apple
```

and each word has a two-dimensional vector.

Initially:

```
king   = [0.2, -0.7]  
queen  = [-0.8, 0.3]  
banana = [0.9, 0.1]  
apple  = [-0.1, -0.5]
```

At this point, these vectors mean nothing.

King might accidentally be closer to banana than queen.

That is completely fine.

Training has not happened yet.

Embeddings do not start meaningful. Their useful structure emerges during training.

24. How Does Training Know What to Learn?

A machine-learning model needs a **task** or **objective**.

A model does not learn meaning simply because we tell it:

"Understand language."

Instead, it learns parameters that help it perform some objective.

Meaningful internal representations can emerge as a consequence.

25. Meaning Through Context

Suppose the training data repeatedly contains:

The king ruled the kingdom.
The queen ruled the kingdom.

The king lived in the palace.
The queen lived in the palace.

The king wore the crown.
The queen wore the crown.

And separately:

I ate a banana.
The banana is a fruit.

The apple is a fruit.
I ate an apple.

Something interesting appears.

king and queen

occur in similar surroundings.

And:

banana and apple

also occur in similar surroundings.

This connects to an important idea in linguistics:

You shall know a word by the company it keeps.

The basic intuition is:

Words appearing in similar contexts often have related meanings or roles.

For example:

```
The king wears a crown.  
The queen wears a crown.  
  
The king lives in a palace.  
The queen lives in a palace.
```

From the model's perspective, `king` and `queen` behave similarly across many contexts.

Therefore, similar internal representations become useful.

Likewise:

```
dog barked  
puppy barked  
  
feed the dog  
feed the puppy
```

The representations of:

```
dog ↔ puppy
```

can become related.

26. A Toy Training Objective

Imagine a very simple task:

Given one word, predict nearby words.

Sentence:

The king wears a crown

Input:

king

Nearby words may include:

the
wears
a
crown

Now consider:

The queen wears a crown

queen appears in almost the same surroundings.

If **king** and **queen** need to produce similar predictions, it becomes useful for their internal representations to become similar.

Before training:

king ●
● queen

After enough training:

king • • queen

Meanwhile:

banana •

may move into a completely different region.

No human explicitly entered:

king is similar to queen

The relationship emerges from patterns in the training data.

27. The Embedding Table

Suppose our vocabulary contains:

king
queen
dog
cat
banana
apple

and every word receives a three-dimensional embedding.

A learned table might look like:

Word	d1	d2	d3
king	0.21	-0.71	0.45
queen	0.19	-0.68	0.49
dog	-0.51	0.22	0.78
cat	-0.48	0.19	0.73
banana	0.89	0.54	-0.13

Word	d1	d2	d3
apple	0.84	0.57	-0.09

This table contains model parameters.

Initially:

```
random numbers
```

After training:

```
useful numbers
```

When the model encounters:

```
dog
```

it can retrieve:

```
[-0.51, 0.22, 0.78]
```

This is commonly called an:

| Embedding lookup

28. How Training Changes the Embeddings

The simplified training cycle is:

1. Start with random vectors.
2. Use them to make a prediction.
3. Compare the prediction with the correct answer.
4. Calculate the error or loss.

5. Adjust the parameters slightly.

6. Repeat again and again.

Example:

Input:

king

Prediction:

banana

Bad prediction.

Loss → high

The parameters are adjusted.

Later:

king → crown

Better prediction.

Over enormous numbers of training examples, the representations gradually organize themselves in ways that help solve the training objective.

Conceptually:

RANDOM SPACE

king

banana

dog

queen

apple

cat

↓ training ↓

LEARNED SPACE

king ● ● queen

dog ● ● cat

banana ● ● apple

29. Does the Machine Discover Features?

In a sense, yes, but we need to be careful with that statement.

In a manually designed vector:

```
[royalty, gender, authority]
```

each coordinate has an obvious human interpretation.

But a learned embedding may look like:

```
king =
[0.31, -0.84, 0.17, 0.66, ...]

queen =
[0.29, -0.78, 0.21, 0.61, ...]
```

It would usually be incorrect to assume:

```
dimension 1 = royalty
dimension 2 = gender
dimension 3 = authority
```

Concepts can be distributed across many dimensions.

For example, something related to royalty may depend on patterns involving:

```
dimension 4
dimension 18
```

```
dimension 91  
dimension 207  
...
```

and those same dimensions may also participate in representing many other concepts.

Meaning usually exists in the overall configuration of the vector rather than one clean human-readable coordinate.

30. Think of Embeddings Like Coordinates on a Map

Consider:

```
Delhi = [28.61, 77.20]
```

Neither number independently represents "Delhi-ness."

```
77.20
```

does not mean:

```
capital city
```

But together, the coordinates place Delhi at a meaningful location in geographic space.

Embeddings follow a similar intuition, except the learned space may contain hundreds or thousands of dimensions.

The location matters because of its relationship with other locations.

31. Why Is This Useful for a Neural Network?

Suppose the system wants to predict:

```
The king lives in the ...
```

Likely possibilities may include:

```
palace  
castle  
kingdom
```

Now consider:

```
The queen lives in the ...
```

Many useful predictions will be similar.

If **king** and **queen** had completely unrelated internal representations, the network would have to learn a lot of similar behavior independently.

But if:

```
king vector  $\approx$  queen vector
```

then related learned machinery can be reused.

This is why semantic structure becomes computationally useful.

The model is not consciously thinking:

```
"King and queen are semantically similar."
```

Instead:

Similar representations are useful because similar inputs often need to produce similar behavior.

32. Latent Dimensions and Latent Space

A useful term is:

Latent

Latent roughly means:

```
hidden
not directly observed
not manually specified
```

In a manually designed vector:

```
[action, romance, comedy]
```

the dimensions are explicit.

In an embedding:

```
[0.21, -0.81, 0.44, ...]
```

the dimensions are learned internal factors.

This is why we often hear terms such as:

- Latent space
- Latent representation
- Latent factors

They refer to learned internal characteristics that are not necessarily human-readable.

33. Vector vs Embedding

This distinction is extremely important.

Every embedding is represented using a vector, but not every vector is an embedding.

For example:

```
[height, weight, age]
```

is a vector.

If those features were manually chosen, we would usually call it a **feature vector**.

An embedding usually refers to:

A learned representation that places an object inside a continuous vector space.

So:

Vector

The mathematical container.

```
[2.1, 4.7, 1.3]
```

Embedding

The learned representation encoded using that vector.

34. Almost Anything Can Be Embedded

Embeddings are not limited to individual words.

```
Word  
↓
```

Vector

Sentence



Vector

Document



Vector

Movie



Vector

User



Vector

Song



Vector

Image



Vector

Product



Vector

The important question is always:

Can we learn a useful mathematical space in which relationships between these objects become accessible through geometry?

35. Embedding Model as a Function

An embedding model can be thought of as a function:

```
Input
  ↓
Embedding Model
  ↓
d-dimensional vector
```

For example:

```
"Batman is a dark superhero movie"
      ↓
    Embedding Model
      ↓
[0.12, -0.37, 0.91, ..., 0.22]
```

If the model produces 768-dimensional embeddings:

```
f(text) →  $\mathbb{R}^{768}$ 
```

If it produces 1536-dimensional embeddings:

```
f(text) →  $\mathbb{R}^{1536}$ 
```

The dimensionality is part of the design of that particular embedding model.

36. What Can We Do Once We Have Embeddings?

Once real-world objects become vectors:

```
Movie → vector  
User  → vector  
Text  → vector  
Image → vector
```

we can perform operations such as:

- Similarity search
- Nearest-neighbor search
- Clustering
- Classification
- Ranking
- Recommendation
- Retrieval

This is why embeddings are such a foundational idea in modern AI systems.

They create a bridge:

```
Human concepts  
  ↓  
Vector representation  
  ↓  
Mathematics  
  ↓  
Machine-learning applications
```

37. The Complete Journey

We started with a simple problem:

Batman

A computer can store the word but does not automatically understand the concept behind it.

We therefore needed numbers.

Our first attempt was a manually created representation:

```
Batman = [  
    Action = 9,  
    Romance = 2,  
    Darkness = 9  
]
```

That gave us a meaningful vector space where mathematical similarity became possible.

But manually designing features does not scale.

Humans cannot define every useful characteristic of:

- Every word
- Every sentence
- Every movie
- Every image
- Every product
- Every person

So instead of asking:

| "Which dimensions should we manually create?"

we changed the question to:

| **"What task should the system learn to solve?"**

Then training adjusts the model's parameters until useful internal representations emerge.

The complete progression becomes:

REAL-WORLD OBJECT

Batman

↓

Need numerical representation

↓

MANUAL VECTOR

[Action=9, Romance=2, Darkness=9]

↓

Does not scale

↓

Let the model learn useful representations

↓

Training objective

+

training data

+

optimization / gradient descent

↓

LEARNED VECTOR

[0.23, -0.81, 0.47, ...]

↓

EMBEDDING

Final Definition

An embedding is not the meaning itself. It is a position inside a learned mathematical space where useful relationships in meaning or behavior become relationships in geometry.

That single idea explains why vectors and embeddings are so important to modern recommendation systems, semantic search, retrieval systems, machine learning, and modern LLM-based applications.